ALKendi
CENTRE DE PRÉPARATION BTS

Développement Mobile

TP10

MOHAMED AMINE BARAKAT
ROUMAISAA BOUDINA

21 Fevrier 2026

2025/2026

hello
- barakat

Enter your message here
Author
ADD

https://amine-6cceb-default-rtdb.firebaseio.com
https://amine-6cceb-default-rtdb.firebaseio.com/
msg
-OllyLrQv3BjO93Uz8as
author: "barakat"
key: "-OllyLrQv3BjO93Uz8as"
msg: "hello"

Partie 1 : Connexion à Firebase

Pour obtenir une instance de Firebase Realtime Database, on utilise la méthode `FirebaseDatabase.getInstance()`.

Pour accéder au nœud nommé **msg**, on utilise une référence obtenue avec `getReference("msg")`.

Partie 2 : Classe modèle Message

La classe Message représente la structure d'un message stocké dans Firebase.
Elle contient trois attributs : le texte du message, la clé unique et l'auteur.

Un constructeur vide est ajouté afin de permettre à Firebase de recréer automatiquement les objets lors de la lecture des données depuis la base.
Un constructeur complet permet d'initialiser la clé, le message et l'auteur.
Des getters et setters sont utilisés pour accéder et modifier les attributs de la classe.

Pourquoi le constructeur vide est-il obligatoire ?
Le constructeur vide est obligatoire car Firebase utilise la réflexion Java pour instancier les objets lors de la récupération des données depuis la base de données.

Partie 3 : Écriture dans la base

L'ajout d'un message dans Firebase se fait en plusieurs étapes.
Tout d'abord, une référence vers le nœud **msg** est obtenue.
Ensuite, la méthode `push()` est utilisée pour générer une clé unique.
Un objet Message est alors créé avec cette clé, le texte du message et l'auteur.
Enfin, la méthode `setValue()` permet d'enregistrer le message dans Firebase.

Partie 4 : Lecture des données

La lecture des messages se fait à l'aide d'un `ValueEventListener`.
Lorsque les données changent dans Firebase, la méthode `onDataChange()` est automatiquement appelée.
La liste des messages est vidée puis remplie à nouveau avec les données récupérées.
Après cela, l'adaptateur est notifié afin de mettre à jour l'affichage du RecyclerView.

Partie 5 : RecyclerView

La méthode `onCreateViewHolder()` permet de créer la vue représentant un message.
La méthode `onBindViewHolder()` associe les données du message à la vue correspondante.
La méthode `getItemCount()` retourne le nombre total de messages à afficher.

Partie 6 : MainActivity

Dans l'activité principale, la liste des messages est initialisée.
Un adaptateur est ensuite créé à partir de cette liste.
Le RecyclerView utilise un `LinearLayoutManager` pour afficher les messages verticalement.
Enfin, l'adaptateur est associé au RecyclerView.

Partie 7 : Suppression d’un message

La suppression d’un message est déclenchée par un appui long sur un élément du RecyclerView.
À l’aide de la clé unique du message, Firebase supprime le nœud correspondant avec la méthode `removeValue()`.
La mise à jour de l’interface est automatique grâce au `ValueEventListener`.

Partie 8 : Modification d’un message

La modification d’un message consiste à changer son contenu en ajoutant la mention **"(modifié)"**.
Le message modifié est ensuite réenregistré dans Firebase avec la méthode `setValue()`.
La clé du message reste inchangée, ce qui garantit la modification du bon message.

Question finale de réflexion

La méthode **push()** permet de générer une clé unique automatiquement.
La méthode **child()** permet d’accéder à un nœud précis de la base de données.
La méthode **setValue()** permet d’écrire ou de mettre à jour des données dans Firebase.
La méthode **removeValue()** permet de supprimer un nœud et son contenu.
La méthode **notifyDataSetChanged()** permet de rafraîchir l’affichage du RecyclerView après un changement de données.